

**Department of
Artificial Intelligence & Machine Learning**

B.Tech III Semester

Academic Year

2026-2027

**Data Base Management Systems
(25B11CS001)**

Capstone Stone Project Documentation

ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437



HOSPITAL MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award
of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

K RAMA DEVI – 25B11AI615

P VAISHNAVI – 25B11AIC35

A BALA TRIPURA SUNDARI – 25B11AI020

B BHARGAVI – 25B11AI166

Under the esteemed guidance of

Mrs. G.L. Narasamba Vanguri, M. Tech, (Ph.D.)

Assistant Professor, Department of AIML



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Hospital Management System” is being submitted by K Ramadevi(Roll No. 25B11AI615), P Vaishnavi (Roll No. 25B11AIC35), A Bala Tripura Sundari (Roll No. 25B11AI020) and B Bhargavi (Roll No. 25B11AI166) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. G.L.Narasamba Vanguri, M. Tech, (Ph.D.)
Assistant professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Hospital Management System” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mrs G.L. Narasamba Vanguri , Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

K.RAMADEVI(25B11AI615)

P.VAISHNAVI(25B11AIC35)

A .B.TSUNDARI(25B11AI02

0) B.BHARGAVI

(25B11AI166)

Date: 14 September 2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

B.BHARGAVI

(25B11AI924) G JASWANTH

(25B11AI333) CH ESWAR

(25B11AI186) CH CHARAN

(25B11AI223)

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: Bank Account Management System

Submitted by:

S.No	Student Name	Roll Number
1	K.RAMADEVI	25B11AI615
2	P.VAISHNAVI	25B11AIC35
3	A.B.T SUNDARI	25B11AI020
4	B.BHARGAVI	25B11AI166

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27 Date of Submission: 14 September 2026

GitHub Repository Link: <https://github.com/DBMS-2026/Hospital-Management-System-DBMS>

1. ABSTRACT

The Hospital Management System is a database project designed to manage hospital information efficiently. It stores and organizes details of patients, doctors, departments, appointments, admissions, treatments, medicines, and bills. The system uses primary and foreign keys to maintain relationships and data integrity. It reduces manual work, avoids data redundancy, and provides easy access to accurate hospital records.

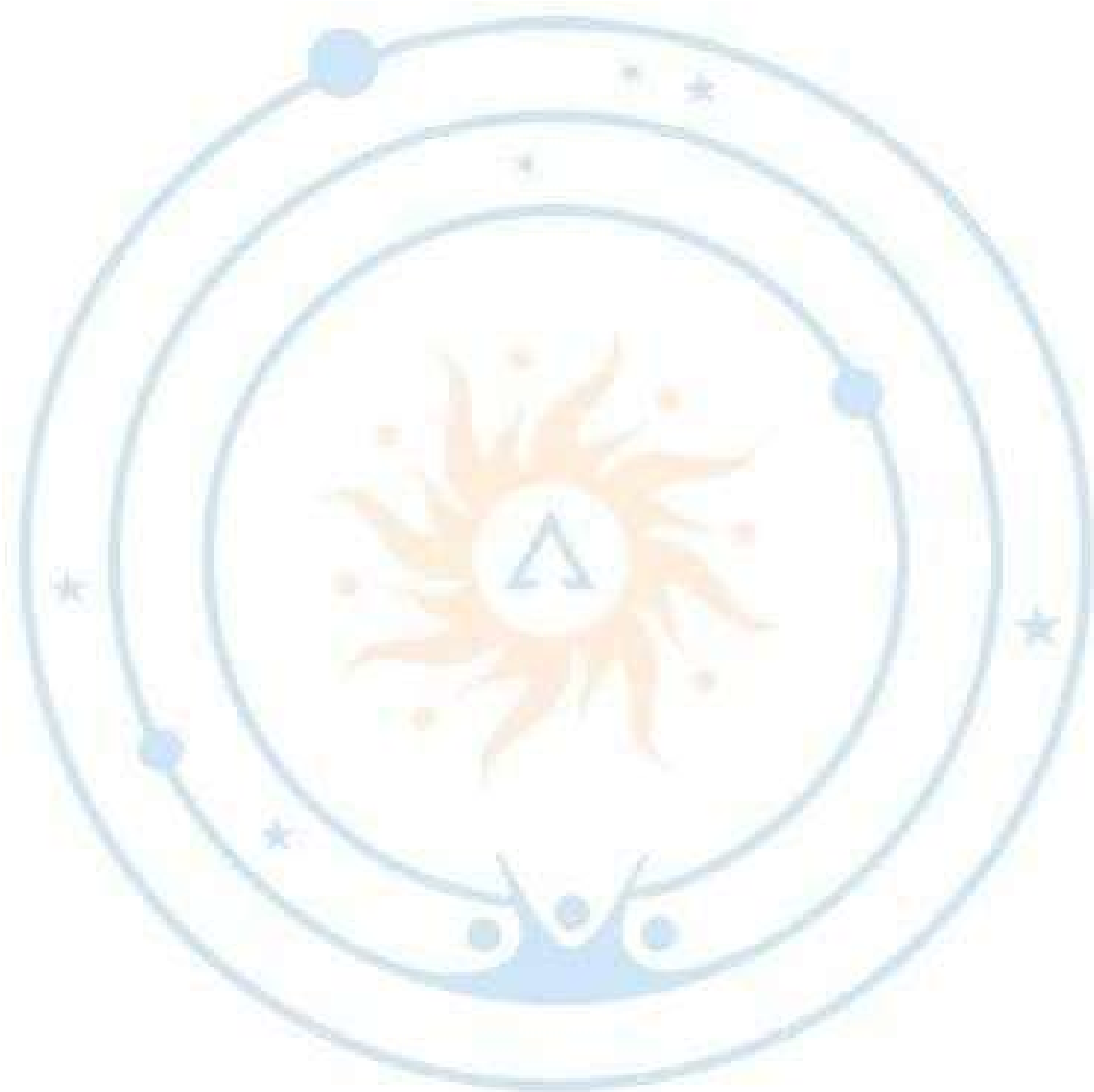


TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	ABSTRACT	v
	List of Figures	vii
	List of Tables	vii
2	INTRODUCTION	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	PROBLEM STATEMENT	1
4	OBJECTIVES	1
5	TOOLS & TECHNOLOGIES USED	2
6	DATABASE DESIGN	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	DATABASE IMPLEMENTATION	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14
10	ENHANCEMENTS & FUTURE IMPROVEMENTS	15
11	CONCLUSION	15
12	REFERENCES	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	4
Figure 1	Customer records / basic SELECT output	12
Figure 2	Active savings accounts / filtering output	12
Figure 3	Aggregate or GROUP BY output	13
Figure 4	Join or subquery output	13

LIST OF TABLES

Table No.	Title of the Table	Page No.
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	2
6.2	Entity Attributes	2
6.4	Relationships & Cardinality	4
6.5	Relational Database Schema	4
7.2	Sample Data — DML.	10
7.3	Constraints & Data Integrity	10
7.4	SQL Operations & Queries	10
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14

2. INTRODUCTION

2.1 Background

Hospital management systems handle large amounts of structured information related to patients, doctors, departments, appointments, admissions, treatments, medicines, and bills. A relational database is required to store this information consistently and to maintain relationships between the different records. In this project, the hospital domain is represented using eight related tables: Patient, Doctor, Department, Appointment, Admission, Treatment, Medicine, and Bill. The schema separates data into appropriate entities and uses keys and constraints to reduce duplication and maintain data integrity. SQL queries are then used to retrieve, update, aggregate, and manage the stored information..

2.2 Purpose of the Database

The database provides a centralized structure for managing patient details, doctor information, department details, appointment schedules, admission records, treatment details, medicine information, and billing records. It supports hospital staff and administrators who need to retrieve patient and doctor information, schedule appointments, manage admissions, record treatments, track medicines, and maintain billing details. The relationships between the tables allow related information to be accessed through SQL joins and subqueries instead of maintaining disconnected records.

3. PROBLEM STATEMENT

Traditional hospital record keeping can lead to **duplicate patient information, inconsistent records and difficulty in managing appointments, treatments and billing.**

The proposed system centralizes patient, doctor, department, appointment, admission, treatment, medicine and billing information into a **single relational database.**

The system provides structured data management, referential integrity, validation and SQL-based queries for efficient hospital operations.

4. OBJECTIVES

Objective 1: Design a relational schema containing eight related hospital entities.

Objective 2: Implement primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints to maintain data integrity.

Objective 3: Populate the database with representative patient, doctor, department, appointment, admission, treatment, medicine, and bill records.

Objective 4: Demonstrate SQL retrieval, updates, joins, aggregate functions, GROUP BY/HAVING, and subqueries.

Objective 5: Document the database design, implementation, sample data, SQL queries, and outputs for the Hospital Management System.

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used
Database Server	MySQL
Query / Admin Tool	MySQL workbench /SQL Environment
Front-end (if any)	Not specified in the SQL implementation
Operating System	Windows
Version Control	GitHub
Database language	SQL
Database Type	Relational Database

6. DATABASE DESIGN

6.1 Identified Entities

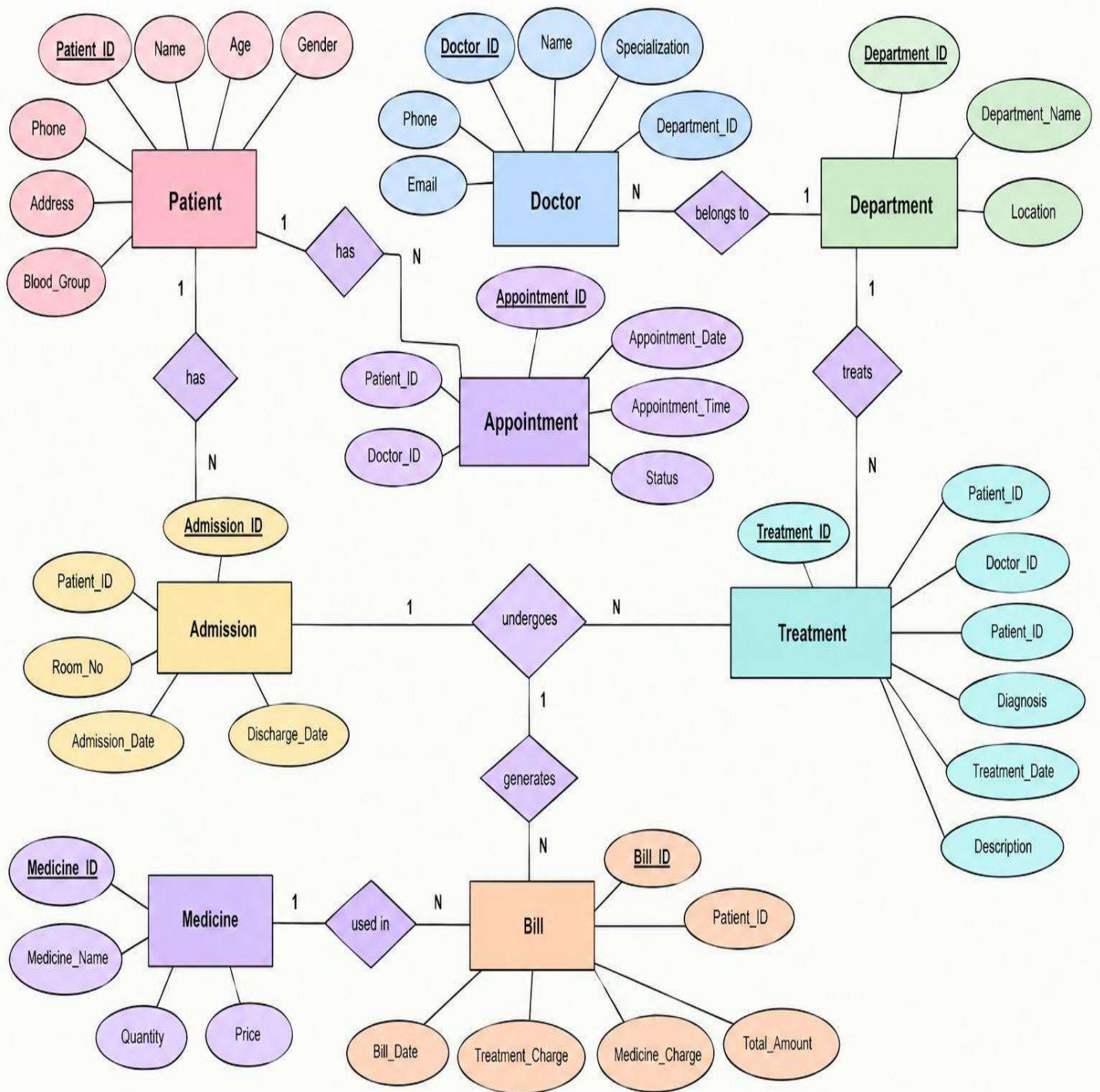
S.No	Entity Name	Description
1	Patient	Stores patient details such as patient ID, name age gender Phone,address, and bloos group.
2	Doctor	Stores doctor details including doctor ID, name, specialization, phone, email, and department.
3	Department	Stores hospital department information such as department ID, department name, and location.
4	Appointment	Stores appointment details including patient, doctor, appointment date, appointment time, and status.
5	Admission	Stores patient admission details such as admission ID, patient, room number, admission date, and discharge date.
6	Treatment	Stores treatment information including patient, doctor, diagnosis, treatment date, and description.
7	Medicine	Stores medicine details such as medicine ID, medicine name, quantity, and price.
8	Bill	Stores patient billing details including bill ID, patient, bill date, treatment charge, medicine charge, and total amount.

6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Patient	Patient_ID	INT	PK	Simple
Patient	Name	VARCHAR(50)	—	Simple
Patient	Age	INT	—	Simple
Patient	Gender	VARCHAR(10)	—	Simple
Patient	Phone	VARCHAR(15)	—	Simple
Patient	Address	VARCHAR(100)	—	Simple
Patient	Blood_Group	VARCHAR(5)	—	Simple
Doctor	Doctor_ID	INT	PK	Simple
Doctor	Name	VARCHAR(50)	—	Simple
Doctor	Specialization	VARCHAR(50)	—	Simple
Doctor	Phone	VARCHAR(15)	—	Simple
Doctor	Email	VARCHAR(100)	—	Simple
Doctor	Department_ID	INT	FK	Simple
Department	Department_ID	INT	PK	Simple
Department	Department_Name	VARCHAR(50)	—	Simple
Department	Location	VARCHAR(100)	—	Simple
Appointment	Appointment_ID	INT	PK	Simple
Appointment	Patient_ID	INT	FK	Simple
Appointment	Doctor_ID	INT	FK	Simple
Appointment	Appointment_Date	DATE	—	Simple
Appointment	Appointment_Time	TIME	—	Simple
Appointment	Status	VARCHAR(20)	—	Simple
Admission	Admission_ID	INT	PK	Simple
Admission	Patient_ID	INT	FK	Simple
Admission	Room_No	VARCHAR(10)	—	Simple
Admission	Admission_Date	DATE	—	Simple
Admission	Discharge_Date	DATE	—	Simple
Treatment	Treatment_ID	INT	PK	Simple
Treatment	Patient_ID	INT	FK	Simple
Treatment	Doctor_ID	INT	FK	Simple
Treatment	Diagnosis	VARCHAR(100)	—	Simple
Treatment	Treatment_Date	DATE	—	Simple
Treatment	Description	VARCHAR(255)	—	Simple
Medicine	Medicine_ID	INT	PK	Simple
Medicine	Medicine_Name	VARCHAR(100)	—	Simple
Medicine	Quantity	INT	—	Simple

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Medicine	Price	DECIMAL(10,2)	—	Simple
Bill	Bill_ID	INT	PK	Simple
Bill	Patient_ID	INT	FK	Simple
Bill	Bill_Date	DATE	—	Simple
Bill	Treatment_Charge	DECIMAL(10,2)	—	Simple
Bill	Medicine_Charge	DECIMAL(10,2)	—	Simple
Bill	Total_Amount	DECIMAL(10,2)	—	Derived

6.3 ER Diagram



6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Patient	has	Appointment	1:N
Doctor	attends	Appointment	1:N
Department	has	Doctor	1:N
Patient	has	Admission	1:N
Patient	receives	Treatment	1:N
Doctor	provides	Treatment	1:N
Patient	receives	Bill	1:N
Medicine	Included in	Bill	1:N

6.5 Relational Database Schema

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Patient	Patient_ID	INT	PK	PRIMARY KEY, NOT NULL
Patient	Name	VARCHAR(50)	—	NOT NULL
Patient	Age	INT	—	NOT NULL, CHECK Age >= 0
Patient	Gender	VARCHAR(10)	—	NOT NULL, CHECK Male/Female/Other
Patient	Phone	VARCHAR(15)	—	NOT NULL
Patient	Address	VARCHAR(100)	—	NOT NULL
Patient	Blood_Group	VARCHAR(5)	—	NOT NULL
Doctor	Doctor_ID	INT	PK	PRIMARY KEY, NOT NULL
Doctor	Name	VARCHAR(50)	—	NOT NULL
Doctor	Specialization	VARCHAR(50)	—	NOT NULL
Doctor	Phone	VARCHAR(15)	—	NOT NULL
Doctor	Email	VARCHAR(100)	—	NOT NULL, UNIQUE
Doctor	Department_ID	INT	FK	NOT NULL, FOREIGN KEY → Department
Department	Department_ID	INT	PK	PRIMARY KEY, NOT NULL
Department	Department_Name	VARCHAR(50)	—	NOT NULL, UNIQUE
Department	Location	VARCHAR(100)	—	NOT NULL
Appointment	Appointment_ID	INT	PK	PRIMARY KEY, NOT NULL
Appointment	Patient_ID	INT	FK	NOT NULL, FOREIGN KEY → Patient
Appointment	Doctor_ID	INT	FK	NOT NULL, FOREIGN KEY → Doctor
Appointment	Appointment_Date	DATE	—	NOT NULL
Appointment	Appointment_Time	TIME	—	NOT NULL
Appointment	Status	VARCHAR(20)	—	NOT NULL, CHECK allowed statuses
Admission	Admission_ID	INT	PK	PRIMARY KEY, NOT NULL
Admission	Patient_ID	INT	FK	NOT NULL, FOREIGN KEY → Patient
Admission	Room_No	VARCHAR(10)	—	NOT NULL
Admission	Admission_Date	DATE	—	NOT NULL
Admission	Discharge_Date	DATE	—	NULL allowed

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Treatment	Treatment_ID	INT	PK	PRIMARY KEY, NOT NULL
Treatment	Patient_ID	INT	FK	NOT NULL, FOREIGN KEY → Patient
Treatment	Doctor_ID	INT	FK	NOT NULL, FOREIGN KEY → Doctor
Treatment	Diagnosis	VARCHAR(100)	—	NOT NULL
Treatment	Treatment_Date	DATE	—	NOT NULL
Treatment	Description	VARCHAR(255)	—	NULL allowed
Medicine	Medicine_ID	INT	PK	PRIMARY KEY, NOT NULL
Medicine	Medicine_Name	VARCHAR(100)	—	NOT NULL, UNIQUE
Medicine	Quantity	INT	—	NOT NULL, CHECK Quantity >= 0
Medicine	Price	DECIMAL(10,2)	—	NOT NULL, CHECK Price >= 0
Bill	Bill_ID	INT	PK	PRIMARY KEY, NOT NULL
Bill	Patient_ID	INT	FK	NOT NULL, FOREIGN KEY → Patient
Bill	Bill_Date	DATE	—	NOT NULL, DEFAULT CURRENT_DATE
Bill	Treatment_Charge	DECIMAL(10,2)	—	NOT NULL, CHECK >= 0
Bill	Medicine_Charge	DECIMAL(10,2)	—	NOT NULL, CHECK >= 0
Bill	Total_Amount	DECIMAL(10,2)	—	NOT NULL, CHECK >= 0

6.6 Normalization

UNF: Hospital information could be represented as a large combined record containing patient, doctor, department, appointment, admission, treatment, medicine, and billing details.

1NF: The design separates repeating groups and keeps attributes atomic. For example, patient details such as Name, Age, Gender, Phone, Address, and Blood_Group are stored as individual attributes.

2NF: Each entity is separated into its own table with a primary key so that non-key attributes depend on the complete primary key. For example, Patient, Doctor, Appointment, Admission, Treatment, Medicine, and Bill are maintained as separate tables.

3NF: Relationships are represented using foreign keys, so patient, doctor, department, appointment, admission, treatment, medicine, and billing information are stored in their appropriate tables rather than being repeatedly duplicated. This reduces update, insertion, and deletion anomalies and improves data integrity.

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

```
-- Drop tables if they exist
DROP TABLE IF EXISTS TransactionRecord;
DROP TABLE IF EXISTS Loan;
DROP TABLE IF EXISTS Account;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Customer;
DROP TABLE IF EXISTS Branch;

-----
-- 1. BRANCH TABLE
-----

CREATE TABLE Branch (
    BranchID INTEGER PRIMARY KEY,
    BranchName VARCHAR(50) NOT NULL,
    IFSCCode VARCHAR(11) NOT NULL UNIQUE,
    City VARCHAR(50) NOT NULL,
    State VARCHAR(50) NOT NULL,
    ManagerName VARCHAR(50) NOT NULL,
    CreatedAt DATE DEFAULT CURRENT_DATE,
    CONSTRAINT chk_ifsc_len CHECK (LENGTH(IFSCCode) = 11)
);

-----
-- 2. CUSTOMER TABLE (Atomic attributes, 1NF/2NF/3NF compliant)
-----

CREATE TABLE Customer (
    CustomerID INTEGER PRIMARY KEY,
    FirstName VARCHAR(30) NOT NULL,
    MiddleName VARCHAR(30),
    LastName VARCHAR(30) NOT NULL,
    DOB DATE NOT NULL,
    Gender VARCHAR(10) CHECK (Gender IN ('Male', 'Female', 'Other')),
    AadhaarNo VARCHAR(12) NOT NULL UNIQUE,
    PANNo VARCHAR(10) NOT NULL UNIQUE,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(12) NOT NULL,
    Address VARCHAR(100) NOT NULL,
    CreatedAt DATE DEFAULT CURRENT_DATE,
    CONSTRAINT chk_aadhaar CHECK (LENGTH(AadhaarNo) = 12),
    CONSTRAINT chk_pan CHECK (LENGTH(PANNo) = 10)
);

-----
-- 3. EMPLOYEE TABLE
-----

CREATE TABLE Employee (
    EmployeeID INTEGER PRIMARY KEY,
    BranchID INTEGER NOT NULL,
    FirstName VARCHAR(30) NOT NULL,
    MiddleName VARCHAR(30),
    LastName VARCHAR(30) NOT NULL,
    Designation VARCHAR(50) NOT NULL,
    Qualifications TEXT,
    Salary DECIMAL(12, 2) NOT NULL CHECK (Salary >= 0),
    Phone VARCHAR(12) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    HireDate DATE NOT NULL,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);
```

```

-----
-- 4. ACCOUNT TABLE
-----
CREATE TABLE Account (
    AccountNumber VARCHAR(20) PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    BranchID INTEGER NOT NULL,
    AccountType VARCHAR(20) NOT NULL CHECK (AccountType IN ('Savings', 'Current',
'Fixed Deposit')),
    Balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (Balance >= 0),
    OpenDate DATE NOT NULL,
    Status VARCHAR(15) NOT NULL DEFAULT 'Active' CHECK (Status IN ('Active',
'Inactive', 'Frozen', 'Closed')),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);

-----
-- 5. TRANSACTION TABLE
-----
CREATE TABLE TransactionRecord
( TransactionID INTEGER PRIMARY KEY,
  AccountNumber VARCHAR(20) NOT NULL,
  TransactionType VARCHAR(15) NOT NULL CHECK (TransactionType IN ('Deposit',
'Withdrawal', 'Transfer')),
  Amount DECIMAL(15, 2) NOT NULL CHECK (Amount > 0),
  TransactionDate DATE DEFAULT CURRENT_DATE,
  Mode VARCHAR(20) NOT NULL CHECK (Mode IN ('Online', 'ATM', 'Branch', 'UPI', 'Net
Banking')),
  Description VARCHAR(255),
  FOREIGN KEY (AccountNumber) REFERENCES Account(AccountNumber)
);

-----
-- 6. LOAN TABLE
-----
CREATE TABLE Loan (
    LoanID INTEGER PRIMARY KEY,
    CustomerID INTEGER NOT NULL,
    BranchID INTEGER NOT NULL,
    LoanType VARCHAR(50) NOT NULL CHECK (LoanType IN ('Home', 'Car', 'Personal',
'Education', 'Business')),
    LoanAmount DECIMAL(15, 2) NOT NULL CHECK (LoanAmount > 0),
    PrincipalAmount DECIMAL(15, 2) CHECK (PrincipalAmount > 0),
    InterestRate DECIMAL(5, 2) NOT NULL CHECK (InterestRate >= 0 AND InterestRate <=
100),
    StartDate DATE NOT NULL,
    DurationMonths INTEGER NOT NULL CHECK (DurationMonths > 0),
    EmiAmount DECIMAL(10, 2) CHECK (EmiAmount >= 0),
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (Status IN ( 'Approved',
'Pending', 'Closed', 'Rejected', 'Active', 'Default')),
    SanctionDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);

```

7.2 Sample Data — DML.

Table	No. of Records	Sample Data / Screenshot Reference
Patient	10	dml.sql – Patient seed records
Doctor	8	dml.sql – Doctor seed records
Department	5	dml.sql – Department seed records
Appointment	15	dml.sql – Appointment seed records
Admission	8	dml.sql – Admission seed records
Treatment	15	dml.sql – Treatment seed records
Medicine	10	dml.sql – Medicine seed records
Bill	10	dml.sql – Bill seed records

7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All eight tables	Uniquely identifies each record and prevents duplicate key values.	ddl.sql
FOREIGN KEY	Doctor, Appointment, Admission, Treatment, Bill	Maintains valid relationships between related tables.	ddl.sql
NOT NULL	Required fields across tables	Prevents missing values in required attributes.	ddl.sql
UNIQUE	Department.Department_Name; Doctor.Email; Medicine.Medicine_Name	Prevents duplicate department names, doctor emails, and medicine names.	ddl.sql
CHECK	Patient.Age, Patient.Gender, Appointment.Status, Medicine.Quantity, Medicine.Price, Bill charges	Restricts invalid or out-of-range values.	ddl.sql
DEFAULT	Appointment.Status	Provides default values when not explicitly supplied.	ddl.sql

7.4 SQL Operations & Queries

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	Basic SELECT — all customers	Retrieve customer records.	queries(2).sql Q1
2	Filtered SELECT — active savings accounts	Find active savings accounts ordered by balance.	queries(2).sql Q2
3	LIKE filtering	Search customers by name or email domain.	queries(2).sql Q3
4	Balance lookup	Check the balance of account 10001.	queries(2).sql Q4
5	UPDATE + INSERT	Deposit ₹5,000 into account 10001 and record the transaction.	queries(2).sql Q5
6	UPDATE + INSERT	Withdraw ₹1,000 when sufficient balance exists and record the transaction.	queries(2).sql Q6
7	Transaction history	Display account transaction history in descending date order.	queries(2).sql Q7
8	SUM aggregate	Calculate total funds across all accounts.	queries(2).sql Q8

S. No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	Basic SELECT — all patients	Retrieve patient records.	queries(2).sql Q1
2	Filtered SELECT — female patients	Find female patients ordered by age.	queries(2).sql Q2
3	LIKE filtering	Search patients by name or address.	queries(2).sql Q3
4	Patient lookup	Check the details of patient 1001.	queries(2).sql Q4
5	UPDATE + INSERT	Update patient details and record a new appointment.	queries(2).sql Q5
6	UPDATE + INSERT	Complete an appointment and record the treatment.	queries(2).sql Q6
7	Treatment history	Display patient treatment history in descending date order.	queries(2).sql Q7
8	SUM aggregate	Calculate total billing amount across all patients.	queries(2).sql Q8
9	GROUP BY + aggregates	Calculate patient count, average age, and total patients by gender.	queries(2).sql Q9
10	GROUP BY + aggregates	Calculate doctor count, total doctors, and average doctors by department.	queries(2).sql Q10
11	JOIN + GROUP BY + HAVING	Find departments whose total doctor count exceeds 1.	queries(2).sql Q11
12	JOIN + GROUP BY + HAVING	Find patients holding more than one appointment.	queries(2).sql Q12
13	Subquery	Find patients whose total bill amount exceeds the average bill amount.	queries(2).sql Q13
14	Subquery + ORDER BY + LIMIT	List treatments for the patient with the highest bill amount.	queries(2).sql Q14

7.5 Sample Outputs / Screenshots

Figure 1: List of Tables in the hospital_db Database

334 • **SHOW TABLES;**

335

Result Grid | Filter Rows: | Export

	Tables_in_hospital_db
►	admission
	appointment
	bill
	department
	doctor
	medicine
	patient
	treatment

Figure 2: Patient Table with Inserted Records

```
34 CREATE TABLE Patient (  
35     Patient_ID INT PRIMARY KEY,  
36     Name VARCHAR(50) NOT NULL,  
37     Age INT NOT NULL CHECK (Age >= 0),  
38     Gender VARCHAR(10) NOT NULL  
39     CHECK (Gender IN ('Male', 'Female', 'Other')),  
40     Phone VARCHAR(15) NOT NULL,  
41     Address VARCHAR(100) NOT NULL,  
42     Blood_Group VARCHAR(5) NOT NULL  
43 );
```

Result Grid

	Patient_ID	Name	Age	Gender	Phone	Address	Blood_Group
▶	1001	Ravi Kumar	45	Male	9876543210	Guntur	A+
	1002	Priya Sharma	32	Female	9876543211	Vijayawada	B+
	1003	Arjun Reddy	55	Male	9876543212	Hyderabad	O+
	1004	Anjali Rao	28	Female	9876543213	Rajahmundry	AB+
	1005	Suresh Kumar	60	Male	9876543214	Kakinada	A-
	1006	Lakshmi Devi	40	Female	9876543215	Guntur	O+
	1007	Rahul Verma	25	Male	9876543216	Visakhapatnam	B-
	1008	Sneha Reddy	35	Female	9876543217	Eluru	A+
	1009	Kiran Kumar	50	Male	9876543218	Tenali	AB-
	1010	Divya Rao	22	Female	9876543219	Vijayawada	O-
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3: Department Table with Inserted Records

```
49  
50 CREATE TABLE Department (  
51     Department_ID INT PRIMARY KEY,  
52     Department_Name VARCHAR(50) NOT NULL UNIQUE,  
53     Location VARCHAR(100) NOT NULL  
54 );  
55
```

Result Grid

	Department_ID	Department_Name	Location
▶	101	Cardiology	Block A
	102	Neurology	Block B
	103	Orthopedics	Block C
	104	Pediatrics	Block D
	105	General Medicine	Block E
*	NULL	NULL	NULL

Figure 4: Doctor Table with Inserted Records

```

61 CREATE TABLE Doctor (
62     Doctor_ID INT PRIMARY KEY,
63     Name VARCHAR(50) NOT NULL,
64     Specialization VARCHAR(50) NOT NULL,
65     Phone VARCHAR(15) NOT NULL,
66     Email VARCHAR(100) NOT NULL UNIQUE,
67     Department_ID INT NOT NULL,
68
69     CONSTRAINT fk_doctor_department
70     FOREIGN KEY (Department_ID)
71     REFERENCES Department(Department_ID)
72 );
73
74

```

Doctor_ID	Name	Specialization	Phone	Email	Department_ID
201	Dr. Rajesh	Cardiologist	9000000001	rajesh@hospital.com	101
202	Dr. Meena	Neurologist	9000000002	meena@hospital.com	102
203	Dr. Arvind	Orthopedic	9000000003	arvind@hospital.com	103
204	Dr. Kavya	Pediatrician	9000000004	kavya@hospital.com	104
205	Dr. Sandeep	General Physician	9000000005	sandeep@hospital.com	105
206	Dr. Priyanka	Cardiologist	9000000006	priyanka@hospital.com	101
207	Dr. Naveen	Orthopedic	9000000007	naveen@hospital.com	103
208	Dr. Anusha	General Physician	9000000008	anusha@hospital.com	105
NULL	NULL	NULL	NULL	NULL	NULL

Figure 5: Appointment Table with Inserted Records

```

79 CREATE TABLE Appointment (
80     Appointment_ID INT PRIMARY KEY,
81     Patient_ID INT NOT NULL,
82     Doctor_ID INT NOT NULL,
83     Appointment_Date DATE NOT NULL,
84     Appointment_Time TIME NOT NULL,
85     Status VARCHAR(20) NOT NULL DEFAULT 'Scheduled'
86     CHECK (Status IN ('Scheduled', 'Completed', 'Cancelled')),
87
88     CONSTRAINT fk_appointment_patient
89     FOREIGN KEY (Patient_ID)
90     REFERENCES Patient(Patient_ID),
91
92     CONSTRAINT fk_appointment_doctor
93     FOREIGN KEY (Doctor_ID)
94     REFERENCES Doctor(Doctor_ID)
95 );

```

Appointment_ID	Patient_ID	Doctor_ID	Appointment_Date	Appointment_Time	Status
301	1001	201	2025-01-05	09:00:00	Completed
302	1002	202	2025-01-06	10:00:00	Completed
303	1003	203	2025-01-07	11:00:00	Completed
304	1004	204	2025-01-08	09:30:00	Scheduled
305	1005	205	2025-01-09	10:30:00	Completed
306	1006	206	2025-01-10	11:30:00	Cancelled
307	1007	207	2025-01-11	12:00:00	Scheduled
308	1008	208	2025-01-12	09:00:00	Completed
309	1009	201	2025-01-13	10:00:00	Scheduled
310	1010	205	2025-01-14	11:00:00	Completed
311	1001	206	2025-01-15	09:30:00	Completed
312	1002	202	2025-01-16	10:30:00	Scheduled
313	1003	203	2025-01-17	11:30:00	Completed

Figure 6: Admission Table with Inserted Records

```

119 CREATE TABLE Treatment (
120     Treatment_ID INT PRIMARY KEY,
121     Patient_ID INT NOT NULL,
122     Doctor_ID INT NOT NULL,
123     Diagnosis VARCHAR(100) NOT NULL,
124     Treatment_Date DATE NOT NULL,
125     Description VARCHAR(255),
126
127     CONSTRAINT fk_treatment_patient
128         FOREIGN KEY (Patient_ID)
129         REFERENCES Patient(Patient_ID),
130
131     CONSTRAINT fk_treatment_doctor
132         FOREIGN KEY (Doctor_ID)
133         REFERENCES Doctor(Doctor_ID)
134 );

```

Admission_ID	Patient_ID	Room_No	Admission_Date	Discharge_Date
401	1001	A101	2025-01-05	2025-01-10
402	1002	B202	2025-01-06	2025-01-09
403	1003	C303	2025-01-07	2025-01-12
404	1004	D404	2025-01-08	2025-01-11
405	1005	E505	2025-01-09	2025-01-15
406	1006	A102	2025-01-10	2025-01-13
407	1007	C304	2025-01-11	2025-01-14
408	1008	E506	2025-01-12	2025-01-16
NULL	NULL	NULL	NULL	NULL

Figure 7: Treatment Table with Inserted Records

```

118
119 CREATE TABLE Treatment (
120     Treatment_ID INT PRIMARY KEY,
121     Patient_ID INT NOT NULL,
122     Doctor_ID INT NOT NULL,
123     Diagnosis VARCHAR(100) NOT NULL,
124     Treatment_Date DATE NOT NULL,
125     Description VARCHAR(255),
126
127     CONSTRAINT fk_treatment_patient
128         FOREIGN KEY (Patient_ID)
129         REFERENCES Patient(Patient_ID),
130
131     CONSTRAINT fk_treatment_doctor
132         FOREIGN KEY (Doctor_ID)
133         REFERENCES Doctor(Doctor_ID)
134 );

```

Treatment_ID	Patient_ID	Doctor_ID	Diagnosis	Treatment_Date	Description
501	1001	201	Heart Checkup	2025-01-05	ECG and regular cardiac examination
502	1002	202	Migraine	2025-01-06	Medication and neurological examination
503	1003	203	Knee Pain	2025-01-07	X-ray and orthopedic treatment
504	1004	204	Fever	2025-01-08	General examination and medication
505	1005	205	Diabetes	2025-01-09	Blood sugar monitoring
506	1006	206	Chest Pain	2025-01-10	ECG and cardiac consultation
507	1007	207	Fracture	2025-01-11	Bone examination and treatment
508	1008	208	Cold and Cough	2025-01-12	General medication
509	1009	201	Blood Pressure	2025-01-13	BP monitoring and consultation
510	1010	205	Fever	2025-01-14	General examination
511	1001	206	Hypertension	2025-01-15	Blood pressure treatment
512	1002	202	Headache	2025-01-16	Neurological examination

Figure 8: Medicine Table with Inserted Records

```

140
141 • CREATE TABLE Medicine (
142     Medicine_ID INT PRIMARY KEY,
143     Medicine_Name VARCHAR(100) NOT NULL UNIQUE,
144     Quantity INT NOT NULL CHECK (Quantity >= 0),
145     Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0)
146 );
147
148

```

Medicine_ID	Medicine_Name	Quantity	Price
601	Paracetamol	100	5.00
602	Amoxicillin	80	12.00
603	Ibuprofen	75	8.00
604	Aspirin	120	4.00
605	Metformin	90	10.00
606	Cetirizine	100	6.00
607	Omeprazole	70	9.00
608	Azithromycin	60	15.00
609	Insulin	40	25.00
610	Vitamin D	100	7.00
NULL	NULL	NULL	NULL

Figure 9: Bill Table with Inserted Records

```

153 • CREATE TABLE Bill (
154     Bill_ID INT PRIMARY KEY,
155     Patient_ID INT NOT NULL,
156     Bill_Date DATE NOT NULL,
157     Treatment_Charge DECIMAL(10,2) NOT NULL
158     CHECK (Treatment_Charge >= 0),
159     Medicine_Charge DECIMAL(10,2) NOT NULL
160     CHECK (Medicine_Charge >= 0),
161     Total_Amount DECIMAL(10,2) NOT NULL
162     CHECK (Total_Amount >= 0),
163
164     CONSTRAINT fk_bill_patient
165     FOREIGN KEY (Patient_ID)
166     REFERENCES Patient(Patient_ID)
167 );

```

Bill_ID	Patient_ID	Bill_Date	Treatment_Charge	Medicine_Charge	Total_Amount
701	1001	2025-01-10	5000.00	1000.00	6000.00
702	1002	2025-01-09	3500.00	800.00	4300.00
703	1003	2025-01-12	7000.00	1500.00	8500.00
704	1004	2025-01-11	2500.00	500.00	3000.00
705	1005	2025-01-15	6000.00	1200.00	7200.00
706	1006	2025-01-13	4500.00	900.00	5400.00
707	1007	2025-01-14	8000.00	2000.00	10000.00
708	1008	2025-01-16	3000.00	600.00	3600.00
709	1009	2025-01-13	4000.00	700.00	4700.00
710	1010	2025-01-14	2000.00	400.00	2400.00
NULL	NULL	NULL	NULL	NULL	NULL

Result 5 Result 6 Result 7 Result 8 Result 9 Patient 10 Department 11 Doctor 12 An

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality
1	Patient Management	Stores and organizes patient identification, age, gender, contact, address, and blood group information.
2	Doctor Management	Maintains doctor details, specialization, contact information, email, and department association.
3	Department Management	Stores department identification, department name, and location information.
4	Appointment Management	Manages patient appointments with doctors, including appointment date, time, and status.
5	Admission Management	Records patient admissions, room details, admission dates, and discharge information.
6	Treatment Management	Records patient diagnosis, doctor details, treatment date, and treatment description.
7	Medicine Management	Stores medicine details, available quantity, and price information.
8	Billing Management & Analysis	Stores patient billing details and supports billing analysis using aggregate, grouped, and SQL queries.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: <https://github.com/DBMS-2026/Hospital-Management-System-DBMS>

Student Name	Roll Number	Role / Responsibility	Contribution Status
K.RAMADEVI	25B11AI615	Wrote DDL scripts (tables, keys, constraints)	Completed
P.VAISHNAVI	25B11AIC35	Designed ER diagram & relational schema	Completed
A.B.T.SUNDARI	25B11AI020	Wrote CRUD, Join & Aggregate queries	Completed
B.BHARGAVI	25B11AI166	Compiled documentation & screenshots	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Future improvements can extend the current Hospital Management System with stronger data security, user authentication and role-based access, patient and doctor dashboards, automated billing calculations, appointment reminders, audit logging, stored procedures and triggers where supported, indexing for frequently queried columns, and a front-end interface for hospital staff. These enhancements can improve efficiency, security, reporting, and overall hospital operations. They are proposed future enhancements rather than implemented features in the current submission.

11. CONCLUSION

The Hospital Management System successfully models the main hospital entities and their relationships using a relational database structure. The DDL defines eight tables with primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. The DML script supplies representative records for patients, doctors, departments, appointments, admissions, treatments, medicines, and bills. The query script demonstrates selection and filtering, data updates, appointment and treatment management, joins, aggregate functions, GROUP BY/HAVING, and subqueries. The project therefore meets its core objectives of relational design, data integrity, sample-data population, and practical SQL analysis. The team also divided the work across DDL development, ER/schema design, query development, and documentation/screenshot compilation.

12. REFERENCES

1. Uploaded project DDL script: ddl.sql.
2. Uploaded project DML/sample-data script: dml.sql.
3. Uploaded SQL query script: queries.sql.
4. Aditya University — DBMS Capstone Project documentation template.
5. GitHub repository: Hospital Management System (project repository).

APPENDIX

A. Complete SQL Script

The final SQL implementation is supplied through the uploaded project files:

- ddl.sql — database table definitions, keys, foreign keys, and constraints.
- dml.sql — sample data for all six tables.
- queries.sql — representative SQL operations and analytical queries.

Repository: <https://github.com/gopipeddinti1986-oss/Bank-Account-Management-System.git>

B. Additional Screenshots / Outputs

Additional evidence can include SQL execution screenshots showing table creation, sample data, customer/account retrieval, balance updates, aggregate results, joins, GROUP BY/HAVING results, and subquery outputs. No execution screenshots were included in the uploaded source files, so this section is intentionally left for the team to add its actual screenshots.

